



NextGen

Efficient Python Lecture 07

CERN
STEAM Academy

13 July 2026

Acceleration and Scalable Execution

Jacek Gatlik

email: jacek.gatlik@agh.edu.pl

www: <https://galaxy.agh.edu.pl/~jgatlik/>

Faculty of Physics and Applied Computer Science,
AGH University of Krakow, Poland



Outline

From measured bottlenecks to acceleration decisions

Algorithmic and memory-aware refactoring

Selective JIT compilation with Numba

Concurrent, parallel, and scalable execution

Further reading

Section 1

From measured bottlenecks to acceleration decisions

Acceleration starts after measurement

In the previous lecture, we focused on measuring performance before changing the code. In this lecture, we ask what to do after the bottleneck has been identified.

data → computation → diagnostics → measurement → acceleration decision

The goal is not to collect tricks. The goal is to choose an acceleration strategy that matches the workload.

A good optimization step should answer three questions:

- ▶ what bottleneck are we trying to remove?
- ▶ what scientific result must remain unchanged?
- ▶ what additional complexity are we willing to accept?

Acceleration is useful only when it preserves the scientific meaning of the computation.

What this lecture adds

Measurement tells us where the program spends time or memory. It does not automatically tell us how to fix it.

This lecture introduces a practical decision framework:

algorithm → data layout → vectorization → memory control
→ JIT compilation → parallel execution

The ordering matters. It is usually better to remove unnecessary work before making unnecessary work faster.

For scientific Python, the best solution is often not the most advanced one. It is the simplest change that gives enough improvement while keeping the numerical model transparent.

The performance decision ladder

A useful mental model is a ladder of increasingly intrusive changes.

Level	Typical question
Algorithm	Are we solving the right computational problem?
Data layout	Are arrays shaped and stored conveniently?
Vectorization	Can Python loops become array operations?
Memory control	Are temporaries and copies avoidable?
JIT compilation	Is there a small numerical kernel to compile?
Parallel execution	Are there independent tasks or independent data chunks?

Moving upward on the ladder usually increases complexity. In a real project, it is worth stopping as soon as the code is fast enough for the scientific task.

Bottleneck to strategy

Different bottlenecks require different responses. A profiler result is useful only if it changes our next decision.

Observation	Possible strategy
Cost grows too fast with size	change the algorithm or reduce repeated work
Most time in Python loop	vectorize or JIT-compile the loop
Most time in NumPy call	inspect array sizes, memory layout, and BLAS usage
Large peak memory	avoid temporaries, stream, chunk, or update in place
Many independent runs	use task parallelism, but check data movement
Slow file access	batch I/O, cache, or change data format

The same optimization technique may be excellent for one bottleneck and irrelevant for another.

A small scientific kernel to keep in mind

We will use a simple nonlinear finite-difference residual as a running example. It is small enough to understand, but realistic enough to show several acceleration choices.

$$r_i = -\frac{u_{i-1} - 2u_i + u_{i+1}}{\Delta x^2} + u_i(u_i^2 - 1).$$

```
1 import numpy as np
2
3 def make_problem(n):
4     x = np.linspace(-10.0, 10.0, n)
5     dx = x[1] - x[0]
6     u = np.tanh(x)
7     return x, u, dx
```

This resembles the residuals that appear in boundary-value problems, relaxation methods, time-stepping schemes, and stability calculations.

A clear baseline implementation

A baseline does not have to be fast. It has to be clear, correct, and easy to compare against future versions.

```
1 def residual_baseline(u, dx):
2     r = np.zeros_like(u)
3
4     for i in range(1, u.size - 1):
5         u_xx = (u[i-1] - 2.0*u[i] + u[i+1]) / dx**2
6         r[i] = -u_xx + u[i]*(u[i]**2 - 1.0)
7
8     return r
```

This version exposes the stencil, the boundary convention, and the nonlinear term. It is also slow for large arrays because the loop is executed at Python level.

Before accelerating it, we should keep it as a numerical reference.

What makes this baseline useful

A useful baseline should be simple enough that we can trust it more than the optimized version.

```
1 x, u, dx = make_problem(2001)
2
3 r0 = residual_baseline(u, dx)
4
5 assert r0.shape == u.shape
6 assert np.isfinite(r0).all()
```

Later versions may use vectorization, in-place updates, Numba, or parallel execution. They should still reproduce the same result within a reasonable tolerance.

```
1 # Later:
2 # assert np.allclose(r_fast, r0)
```

Optimization without a reference solution quickly becomes guesswork.

Not every loop is equally important

Scientific programs often contain many loops. Some are responsible for most of the runtime. Others are harmless.

```
1 for parameter in parameter_values:
2     result = solve_problem(parameter)
3     save_summary(parameter, result)
```

This outer loop may look inefficient, but if each call to `solve_problem` is expensive, the loop overhead may be negligible.

A different loop may be executed millions of times inside a numerical kernel:

```
1 for i in range(1, u.size - 1):
2     r[i] = stencil_expression(u, i)
```

Acceleration should focus on hot loops, not on loops that merely organize the workflow.

The cost of complexity

Every optimization has a price.

Technique	Possible cost
Vectorization	temporary arrays, less explicit control flow
In-place updates	harder reasoning about state and old values
JIT compilation	supported subset of Python, compilation time
Processes	serialization and data movement costs
Threads	workload-dependent speedup and synchronization issues
Alternative backends	new API assumptions and portability questions

The right question is not “can this be faster?”. The right question is “is the speedup worth the complexity for this scientific problem?”.

Section 2

Algorithmic and memory-aware refactoring

Algorithm first

The largest speedups often come from changing the amount of work, not from accelerating the same work.

For example, an $\mathcal{O}(n^2)$ algorithm may be acceptable for small data, but impossible for large data. No low-level trick can fully compensate for the wrong scaling.

Size	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$
10^3	10^3 operations	10^6 operations
10^5	10^5 operations	10^{10} operations
10^7	10^7 operations	10^{14} operations

Before accelerating Python, ask whether the numerical method, data structure, or mathematical formulation can be improved.

Scaling can reveal the wrong algorithm

A timing result at one size may hide the real problem. Timing across sizes helps us distinguish constant overhead from algorithmic scaling.

```
1 sizes = [1_000, 3_000, 10_000, 30_000, 100_000]
2
3 for n in sizes:
4     x, u, dx = make_problem(n)
5
6     # Measure the same function for different n
7     # and inspect how the time grows.
8     r = residual_baseline(u, dx)
```

If the time grows roughly linearly, local implementation changes may be enough. If the time grows quadratically or worse, the algorithm itself may be the main bottleneck.

Scaling plots are often more informative than a single benchmark number.

→ **Example 07 - 01**

Reducing unnecessary work

A common algorithmic improvement is to avoid recomputing quantities that do not change.

```
1 # Less efficient: dx**2 is recomputed conceptually each iteration
2 for i in range(1, u.size - 1):
3     u_xx = (u[i-1] - 2.0*u[i] + u[i+1]) / dx**2
4     r[i] = -u_xx + u[i]*(u[i]**2 - 1.0)
5
6 # Better: compute constant factors once
7 inv_dx2 = 1.0 / dx**2
8
9 for i in range(1, u.size - 1):
10    u_xx = (u[i-1] - 2.0*u[i] + u[i+1]) * inv_dx2
11    r[i] = -u_xx + u[i]*(u[i]**2 - 1.0)
```

This is a small example. In real simulations, the same idea may apply to grid coefficients, masks, basis functions, interpolation indices, or geometry factors.

Precompute coefficients, not physics

Precomputation is useful when a quantity is fixed during many calls of a kernel. For example, coefficients in a spatially varying finite-difference operator may depend only on the grid.

```
1  # Coefficients computed once
2  F_half = 0.5 * (F[1:] + F[:-1])
3  inv_dx2 = 1.0 / dx**2
4
5  def residual_with_coefficients(u, F_half, inv_dx2):
6      r = np.zeros_like(u)
7
8      flux_right = F_half[1:] * (u[2:] - u[1:-1])
9      flux_left  = F_half[:-1] * (u[1:-1] - u[:-2])
10
11     r[1:-1] = -(flux_right - flux_left) * inv_dx2
12     r[1:-1] += u[1:-1] * (u[1:-1]**2 - 1.0)
13
14     return r
```

The scientific model is unchanged. Only the repeated bookkeeping has been moved out of the hot path.

Data layout is part of performance

Arrays are not only shapes. They also have a memory layout. Two arrays may have the same shape but very different access patterns.

```
1 A = np.ones((2000, 2000))
2
3 print(A.shape)
4 print(A.strides)
5 print(A.flags["C_CONTIGUOUS"])
```

For C-contiguous NumPy arrays, the last axis varies fastest in memory. Accessing data along contiguous memory is usually more cache-friendly than jumping through memory with a large stride.

This matters in loops, low-level kernels, and some linear algebra workflows.

Strides and contiguous access

A slice may look small and simple, but it can have non-contiguous memory access.

```
1 A = np.ones((1000, 1000))
2
3 row = A[10, :]      # usually contiguous
4 col = A[:, 10]      # usually strided
5
6 print(row.strides)
7 print(col.strides)
```

This does not mean that column slices are wrong. It means that memory access has a direction.

When a kernel loops repeatedly over one axis, it is worth asking whether the array layout matches the dominant access pattern.

A memory-order example

Suppose we store a time series of one-dimensional fields. Two layouts are possible.

```
1 # Option 1: time axis first
2 U1 = np.empty((nt, nx))    # U1[k, i]
3
4 # Option 2: space axis first
5 U2 = np.empty((nx, nt))    # U2[i, k]
```

If we often process one full spatial field at a fixed time, `U1[k, :]` is convenient and contiguous in the default C order.

If we often analyze the full time signal at one spatial point, `U2[i, :]` may be more convenient.

The best layout depends on the operations that dominate the computation.

Loop-to-array refactoring

The most common acceleration step in scientific Python is replacing an elementwise Python loop with an array operation.

```
1 def residual_vectorized(u, dx):  
2     r = np.zeros_like(u)  
3  
4     inv_dx2 = 1.0 / dx**2  
5  
6     u_xx = (u[:-2] - 2.0*u[1:-1] + u[2:]) * inv_dx2  
7     r[1:-1] = -u_xx + u[1:-1]*(u[1:-1]**2 - 1.0)  
8  
9     return r
```

This moves the inner loop from Python into optimized compiled array operations.

For many numerical kernels, this is the simplest and most effective acceleration strategy.

Vectorization is not the same as no loops

Vectorized code still contains loops. They are just executed inside NumPy, usually in compiled code.

```
1 r[1:-1] = (  
2     -(u[:-2] - 2.0*u[1:-1] + u[2:]) / dx**2  
3     + u[1:-1]*(u[1:-1]**2 - 1.0)  
4 )
```

This version is compact and often fast. But it also creates intermediate array-sized values.

For small and medium arrays, this is usually fine. For very large arrays, the limiting factor may become memory bandwidth rather than arithmetic.

Vectorization has a cost model: fewer Python operations, but possibly more temporary arrays.

Temporary arrays are hidden work

A mathematical expression may correspond to several array allocations.

```
1 y = np.sin(u)**2 + np.cos(u)**2
```

This expression may create temporary arrays for `sin(u)`, `cos(u)`, and intermediate powers or sums. The temporaries disappear quickly, but they still consume memory bandwidth and peak memory.

For a field with millions of points, temporary arrays can dominate the runtime.

The question is not whether the expression is mathematically simple. The question is how much data must be moved through memory to evaluate it.

Use output arrays when the operation allows it

Many NumPy universal functions support an `out` argument. This can reduce temporary allocation when memory is the bottleneck.

```
1 tmp = np.empty_like(u)
2 y = np.empty_like(u)
3
4 np.sin(u, out=tmp)      # tmp = sin(u)
5 np.square(tmp, out=tmp) # tmp = sin(u)**2
6
7 np.cos(u, out=y)        # y = cos(u)
8 np.square(y, out=y)     # y = cos(u)**2
9
10 np.add(tmp, y, out=y)   # y = tmp + y
```

This is less elegant than a single expression, so it should be used deliberately. It is most useful when arrays are large and memory traffic has been measured as important.

Streaming instead of storing everything

Sometimes the fastest and most memory-efficient array is the one we never store. If we only need a summary over many snapshots, we do not necessarily need to keep all snapshots in memory.

```
1 total = 0.0
2 count = 0
3
4 for u in load_snapshots():
5     density = energy_density(u)
6     total += density.mean()
7     count += 1
8
9 mean_energy = total / count
```

This pattern is useful for long simulations, detector streams, parameter sweeps, and large data files.

Streaming trades random access for lower memory usage.

→ **Example 07 - 02**

Chunking large calculations

If a vectorized operation is too large to fit comfortably in memory, process the data in chunks.

```
1 def compute_in_chunks(x, block_size):
2     y = np.empty_like(x)
3
4     for start in range(0, x.size, block_size):
5         stop = min(start + block_size, x.size)
6         sl = slice(start, stop)
7
8         y[sl] = np.exp(-x[sl]**2) * np.sin(x[sl])
9
10    return y
```

Chunking is not only for file I/O. It is also useful when a fully vectorized expression would create too many temporary arrays.

The block size becomes a performance parameter and should be chosen empirically.

In-place updates in time stepping

In time-dependent simulations, in-place updates can reduce allocation pressure.

```
1 u = initial_condition(x)
2 r = np.empty_like(u)
3
4 for step in range(nsteps):
5     compute_residual(u, dx, out=r)
6
7     # Intentional in-place update
8     u -= dt * r
```

This is efficient because the arrays `u` and `r` are reused at every time step.

The cost is conceptual: after the update, the old value of `u` is gone. If the old value is needed for diagnostics, adaptive methods, or multi-step schemes, it must be stored deliberately.

Avoid accidental copies

Some indexing operations produce views, while others produce copies. This affects both memory usage and mutation.

```
1 u = np.arange(10.0)
2
3 view = u[2:8]           # usually a view
4 copy = u[[2, 4, 6]]     # fancy indexing creates a copy
5 mask_copy = u[u > 3]    # boolean indexing creates a copy
6
7 print(np.shares_memory(u, view))
8 print(np.shares_memory(u, copy))
9 print(np.shares_memory(u, mask_copy))
```

Copies are not bad. They are sometimes necessary. The problem is an accidental copy inside a large repeated computation.

Memory-aware code makes independence explicit.

Broadcasting wisely

Broadcasting can express many independent computations without explicitly constructing a full grid of repeated values.

```
1 x = np.linspace(-5.0, 5.0, 2000)
2 sigma = np.array([0.5, 1.0, 2.0, 4.0])
3
4 X = x[:, None]           # shape (2000, 1)
5 S = sigma[None, :]       # shape (1, 4)
6
7 U = np.exp(-(X / S)**2)   # shape (2000, 4)
```

Here, broadcasting represents four Gaussian profiles on the same grid.

However, the result `U` is still a real array. Broadcasting avoids some explicit repetition, but the final output may still be large.

When a loop is clearer than a vectorized expression

Vectorization should improve the program, not obscure the numerical method. A compact expression can become difficult to read when it mixes indexing, boundary conditions, masking, coefficients, and nonlinear terms.

```
1 # This may be fast, but it is not always the best teaching code.  
2 r[1:-1] = complicated_expression_with_many_slices
```

A clear loop may be preferable when:

- ▶ the algorithm has many branches,
- ▶ the stencil is irregular,
- ▶ the code is still being debugged,
- ▶ the next step is JIT compilation.

Readable scalar loops and Numba often form a very good combination.

Section 3

Selective JIT compilation with Numba

Selective JIT compilation

Just-in-time compilation is useful when the slow part of the program is a numerical kernel that can be expressed with typed arrays, scalar arithmetic, and simple control flow. The idea is not to compile the whole project.

Python workflow → small numerical kernel → compiled machine code

Keep the surrounding code in ordinary Python:

- ▶ reading input files,
- ▶ setting parameters,
- ▶ choosing experiments,
- ▶ plotting,
- ▶ saving results.

Compile only the part where Python overhead is actually important.

A good Numba target

A good Numba target is usually a small function with a simple numerical interface.

```
1 def kernel(u, dx):  
2     # good candidate:  
3     # - NumPy array input  
4     # - scalar parameters  
5     # - numerical loops  
6     # - no plotting  
7     # - no file I/O  
8     # - no complicated Python objects  
9     ...
```

Numba is especially useful when vectorization would create too many temporaries or when the algorithm is naturally written as a loop.

It is less useful for code dominated by high-level Python objects, pandas operations, plotting, or file handling.

First Numba version

The baseline loop can be compiled with Numba using `@njit`. The code remains close to the original mathematical stencil.

```
1 from numba import njit
2
3 @njit(cache=True)
4 def residual_numba(u, dx):
5     r = np.zeros_like(u)
6     inv_dx2 = 1.0 / dx**2
7
8     for i in range(1, u.size - 1):
9         u_xx = (u[i-1] - 2.0*u[i] + u[i+1]) * inv_dx2
10        r[i] = -u_xx + u[i]*(u[i]**2 - 1.0)
11
12    return r
```

This is often the cleanest use of Numba: keep the algorithm explicit, but remove Python-level loop overhead.

Compilation time is part of the measurement

The first call to a Numba-compiled function includes compilation. Later calls use the compiled version.

```
1 x, u, dx = make_problem(100_000)
2
3 # First call: includes compilation
4 r = residual_numba(u, dx)
5
6 # Later calls: measure execution, not compilation
7 from time import perf_counter
8
9 t0 = perf_counter()
10 for _ in range(100):
11     r = residual_numba(u, dx)
12 t1 = perf_counter()
13
14 print((t1 - t0) / 100)
```

A fair benchmark should separate compilation time from steady-state execution time, unless compilation itself is part of the real workload.

Validate the compiled version

A compiled version should first be treated as a suspect, not as an improvement. It must reproduce the baseline before we trust its speed.

```
1 x, u, dx = make_problem(100_000)
2
3 r_ref = residual_baseline(u, dx)
4 r_jit = residual_numba(u, dx)
5
6 print(np.max(np.abs(r_ref - r_jit)))
7
8 assert np.allclose(r_jit, r_ref)
```

The tolerance should be chosen for the problem. For simple arithmetic kernels, the results may be nearly identical. For more complicated algorithms, small differences may appear because of operation ordering or floating-point effects.

Correctness remains the first performance requirement.

Thinking in nopython mode

The most useful Numba functions are usually those that can be compiled without relying on the Python runtime during execution.

A good mental model is:

typed arrays + scalar variables + numerical loops + simple control flow

```
1 @jit(cache=True)
2 def good_kernel(u, a):
3     s = 0.0
4
5     for i in range(u.size):
6         s += a*u[i]**2
7
8     return s
```

This style may look less “Pythonic” than high-level code. That is often the point: inside a numerical kernel, we want predictable low-level operations.

What not to put inside a JIT kernel

A JIT-compiled kernel should not contain the whole scientific workflow.

```
1 # Usually not a good JIT target
2 def whole_experiment(path, parameters):
3     data = read_file(path)
4     clean = clean_table(data)
5     result = run_solver(clean, parameters)
6     make_plot(result)
7     save_result(result)
8     return result
```

Instead, keep workflow logic in ordinary Python and compile only the numerical core.

```
1 # Better target
2 @jit(cache=True)
3 def solver_step(u, dx, dt, out):
4     ...
```

This separation keeps the project readable and makes the optimized part easier to test.

Using output arrays with Numba

Numba works very naturally with explicit output arrays. This can reduce repeated allocation in time-stepping or iterative algorithms.

```
1 @njit(cache=True)
2 def residual_numba_out(u, dx, r):
3     inv_dx2 = 1.0 / dx**2
4
5     r[0] = 0.0
6     r[-1] = 0.0
7
8     for i in range(1, u.size - 1):
9         u_xx = (u[i-1] - 2.0*u[i] + u[i+1]) * inv_dx2
10        r[i] = -u_xx + u[i]*(u[i]**2 - 1.0)
```

The function modifies `r` intentionally. This is efficient, but it should be documented by the function name or interface.

Compiled time stepping

Once the residual is written into a reusable output array, a simple time-stepping loop can avoid most temporary allocations.

```
1 @jit(cache=True)
2 def evolve(u, dx, dt, nsteps):
3     r = np.empty_like(u)
4
5     for step in range(nsteps):
6         residual_numba_out(u, dx, r)
7
8         for i in range(u.size):
9             u[i] -= dt * r[i]
10
11     return u
```

This is not always the most accurate time integrator. The point is structural: an entire numerical loop can be compiled while keeping allocation and mutation under explicit control.

Parallel loops with Numba

Some loops have independent iterations. In such cases, Numba can parallelize the loop over CPU threads using `parallel=True` and `prange`.

```
1 from numba import njit, prange
2
3 @njit(parallel=True, cache=True)
4 def residual_numba_parallel(u, dx, r):
5     inv_dx2 = 1.0 / dx**2
6
7     r[0] = 0.0
8     r[-1] = 0.0
9
10    for i in prange(1, u.size - 1):
11        u_xx = (u[i-1] - 2.0*u[i] + u[i+1]) * inv_dx2
12        r[i] = -u_xx + u[i]*(u[i]**2 - 1.0)
```

This is appropriate because each value `r[i]` is written independently.

When `prange` is safe

A parallel loop is safe when different iterations do not depend on each other in a hidden way.

Usually safe

- writing `out[i]` independently
- local stencil into a separate output
- independent parameter runs
- reductions supported by the compiler

Usually unsafe

- updating the same scalar without reduction logic
- modifying neighboring elements in place
- time stepping where step $n + 1$ needs step n
- appending to shared Python containers

Parallelism changes the execution order. If the result depends on a particular order of updates, the algorithm is not embarrassingly parallel.

Parallel reductions

Many diagnostics are reductions: sums, norms, energies, averages. These can often be parallelized safely because the operation has a well-defined reduction structure.

```
1 @njit(parallel=True, cache=True)
2 def mean_square(u):
3     s = 0.0
4
5     for i in prange(u.size):
6         s += u[i]**2
7
8     return s / u.size
```

Floating-point reductions may not give bitwise identical results when the order of summation changes. Usually this is acceptable, but it should be understood.

For sensitive calculations, numerical accuracy and reproducibility may matter more than maximum speed.

→ **Example 07 - 03**

Numba is not magic

Numba is powerful, but it is not a universal acceleration button.

It is most useful when:

- ▶ the bottleneck is a numerical loop,
- ▶ the data are NumPy arrays and scalar values,
- ▶ the algorithm is easier to express as loops than as vectorized array expressions,
- ▶ memory temporaries are a problem,
- ▶ the compiled function is called many times or on large data.

It is less useful when the runtime is dominated by file I/O, plotting, pandas transformations, already-optimized linear algebra, or communication between processes.

The best use of Numba is selective.

Section 4

Concurrent, parallel, and scalable execution

Concurrency and parallelism

The words concurrency and parallelism are often used together, but they answer different questions.

Concurrency
Parallelism

managing multiple tasks whose execution overlaps
executing multiple tasks at the same physical time

Concurrency is useful when tasks wait: for files, networks, databases, user input, or external programs.

Parallelism is useful when there is real computational work that can be divided across cores, nodes, or accelerators.

In scientific Python, the best tool depends strongly on the workload.

Workload-dependent choices

Before choosing threads, processes, Numba, or a distributed tool, classify the workload.

Workload	Typical choice	Risk
pure Python CPU loop	process pool or Numba	overhead
NumPy-heavy computation	vectorization, threads, BLAS control	oversubscription
many independent simulations	joblib or process pool	data movement
many small files	batching, threads, async I/O	too many tasks
large arrays shared by tasks	memory mapping or chunking	copies
GPU-suitable array work	alternative backend	transfer cost

There is no best parallel tool. There is only a tool that matches the bottleneck.

The Global Interpreter Lock

In standard CPython, Python bytecode execution is constrained by the Global Interpreter Lock. This means that threads do not automatically make pure Python CPU loops faster.

```
1 # This is unlikely to speed up with ordinary threads
2 def pure_python_sum(values):
3     s = 0.0
4     for x in values:
5         s += x*x
6     return s
```

However, the GIL is not the whole story. Many operations implemented in C, C++, or Fortran may release it. NumPy, SciPy, image processing libraries, and compiled extensions may therefore benefit from threads in some cases.

The practical rule remains: measure the actual workload.

Executors: one interface, different execution models

The `concurrent.futures` module provides a convenient interface for launching tasks.

```
1 from concurrent.futures import ThreadPoolExecutor
2 from concurrent.futures import ProcessPoolExecutor
3
4 # Same basic pattern:
5 # 1. create an executor
6 # 2. submit tasks
7 # 3. collect results
```

The interface is similar, but the execution model is different.

`ThreadPoolExecutor`

multiple threads in one process

`ProcessPoolExecutor`

multiple Python processes

The choice affects memory sharing, serialization, startup overhead, and interaction with the GIL.

Threads for waiting tasks

Threads are often useful when each task spends much of its time waiting. A common example is reading or writing many files.

```
1 from concurrent.futures import ThreadPoolExecutor
2 from pathlib import Path
3
4 def read_one(path):
5     return path.read_text()
6
7 paths = list(Path("data").glob("*.txt"))
8
9 with ThreadPoolExecutor(max_workers=8) as ex:
10     texts = list(ex.map(read_one, paths))
```

This is concurrency more than numerical acceleration. The CPU may not be fully occupied, but the program can make progress on one file while another operation is waiting.

For many small files, batching may still be better than launching thousands of tiny tasks.

Threads for NumPy-heavy work

Threads may also help when each task calls compiled numerical code that releases the GIL.

```
1 from concurrent.futures import ThreadPoolExecutor
2
3 def spectrum(signal):
4     return np.abs(np.fft.rfft(signal))
5
6 signals = [rng.normal(size=200_000) for _ in range(16)]
7
8 with ThreadPoolExecutor(max_workers=4) as ex:
9     spectra = list(ex.map(spectrum, signals))
```

This may or may not be faster than the serial version. It depends on array sizes, memory bandwidth, internal threading of numerical libraries, and the number of physical cores.

Threads are not a replacement for measurement.

Processes for CPU-bound Python work

Separate processes can run Python code in parallel without sharing the same interpreter state.

```
1 from concurrent.futures import ProcessPoolExecutor
2
3 def run_one(parameter):
4     x, u, dx = make_problem(200_000)
5     # Some expensive computation depending on parameter
6     return parameter, np.max(np.abs(parameter * u))
7
8 if __name__ == "__main__":
9     params = np.linspace(0.1, 2.0, 20)
10
11     with ProcessPoolExecutor(max_workers=4) as ex:
12         results = list(ex.map(run_one, params))
```

The `if __name__ == "__main__":` guard is important in scripts, especially on systems where worker processes start fresh Python interpreters.

Processes have data-movement costs

Processes do not share ordinary Python objects in the same way as threads. Arguments and results must usually be serialized and transferred.

```
1 # Potentially expensive:  
2 # each task receives a large array  
3 with ProcessPoolExecutor(max_workers=4) as ex:  
4     results = list(ex.map(run_with_large_array, large_arrays))
```

This can be slower than serial execution if each task is small or if most of the time is spent moving data.

A better strategy is often:

- ▶ pass small parameters,
- ▶ load data inside the worker,
- ▶ use memory mapping for large read-only arrays,
- ▶ group many small operations into one larger task.

→ **Example 07 - 04**

Task granularity

Parallel tasks should be large enough to justify their overhead.

Too fine

millions of tiny tasks, overhead dominates

Too coarse

poor load balancing, some workers idle

Reasonable

each task performs enough useful work

```
1 # Often poor: many tiny tasks
2 tasks = range(1_000_000)
3
4 # Often better: fewer chunks
5 chunks = np.array_split(np.arange(1_000_000), 32)
```

The best chunk size is empirical. It depends on the computation, memory access, and scheduling overhead.

Collecting results as they finish

When tasks have variable duration, it can be useful to process results as soon as they finish.

```
1 from concurrent.futures import ProcessPoolExecutor, as_completed
2
3 params = np.linspace(0.1, 2.0, 20)
4
5 with ProcessPoolExecutor(max_workers=4) as ex:
6     future_to_param = {
7         ex.submit(run_one, p): p for p in params
8     }
9
10    for future in as_completed(future_to_param):
11        p = future_to_param[future]
12        value = future.result()
13        print(p, value)
```

This pattern is useful for parameter sweeps where some parameter values are much more expensive than others.

Asynchronous I/O

Asynchronous I/O is useful when a program manages many waiting operations using an event loop.

It is not primarily a tool for accelerating numerical kernels. It is a tool for coordinating tasks that spend time waiting for external resources.

```
1 # Schematic structure only
2 async def handle_one_request(run_id):
3     data = await read_from_service(run_id)
4     summary = summarize(data)
5     await save_summary(run_id, summary)
```

For a typical CPU-heavy simulation, `asyncio` is usually not the first acceleration strategy. For services, dashboards, remote APIs, or many network requests, it may be the natural model.

Joblib for embarrassingly parallel loops

Many scientific workloads consist of independent runs:

- ▶ different parameter values,
- ▶ different random seeds,
- ▶ different files,
- ▶ different initial conditions,
- ▶ different bootstrap samples.

This is often called embarrassingly parallel computation because the tasks need little or no communication.

$$\begin{array}{l} \text{parameter } p_1 \rightarrow \text{result } r_1 \\ \text{parameter } p_2 \rightarrow \text{result } r_2 \\ \vdots \\ \text{parameter } p_N \rightarrow \text{result } r_N \end{array}$$

This is the situation where `joblib` is especially convenient.

A minimal joblib example

The basic joblib pattern is close to a parallel version of a Python list comprehension.

```
1 from joblib import Parallel, delayed
2
3 def run_one(parameter):
4     x, u, dx = make_problem(200_000)
5     value = np.trapezoid(np.exp(-parameter*x**2), x)
6     return parameter, value
7
8 params = np.linspace(0.1, 2.0, 20)
9
10 results = Parallel(n_jobs=4)(
11     delayed(run_one)(p) for p in params
12 )
```

The code stays readable: define one independent task, then map it over many inputs.

What joblib does not remove

Joblib makes parallel loops convenient, but it does not remove the fundamental costs of parallel execution.

Cost	Meaning
startup	workers must exist before they compute
serialization	arguments and results may need conversion
data movement	large arrays may be copied or transferred
scheduling	tasks must be assigned and collected
memory pressure	several workers may allocate at once

Parallel code can be slower than serial code when tasks are too small or when data movement dominates the calculation.

Large read-only arrays and memory mapping

If many workers need access to the same large read-only array, copying it to every worker may be wasteful.

One common strategy is to store the array in a memory-mapped form and let workers read from it.

```
1 from joblib import dump, load
2
3 u = np.random.default_rng(123).normal(size=10_000_000)
4
5 dump(u, "large_array.joblib")
6 u_shared = load("large_array.joblib", mmap_mode="r")
7
8 # u_shared behaves like an array, but the data can be
9 # accessed through a memory-mapped file.
```

This does not make computation free. It changes the data movement pattern and can reduce memory duplication.

Avoid nested parallelism

A common problem in scientific Python is accidental nested parallelism.

```
1 # Example idea:  
2 # 8 joblib workers  
3 # each worker calls NumPy/SciPy/BLAS using 8 threads  
4 # possible result: 64 active threads
```

More threads than cores can lead to oversubscription: the system spends too much time switching between tasks and moving data.

For controlled benchmarks, it is often useful to set thread limits before importing numerical libraries.

```
1 import os  
2  
3 os.environ["OMP_NUM_THREADS"] = "1"  
4 os.environ["MKL_NUM_THREADS"] = "1"  
5  
6 import numpy as np
```

Reproducible parallel random numbers

Random numbers require extra care in parallel simulations. A good pattern is to create independent child seeds and pass them explicitly to workers.

```
1 def run_random_experiment(parameter, seed_sequence):
2     rng = np.random.default_rng(seed_sequence)
3     noise = rng.normal(size=100_000)
4     return parameter, np.mean(parameter + noise)
5
6 params = np.linspace(0.1, 2.0, 20)
7
8 base_seed = np.random.SeedSequence(12345)
9 child_seeds = base_seed.spawn(len(params))
10
11 results = Parallel(n_jobs=4)(
12     delayed(run_random_experiment)(p, s)
13     for p, s in zip(params, child_seeds)
14 )
```

Reproducibility should not depend on the accidental order in which workers finish.

Shared mutable state is dangerous

Parallel code should avoid hidden shared mutable state. This is especially important with threads, but the design rule is useful everywhere.

```
1 # Fragile design
2 results = []
3
4 def worker(parameter):
5     value = run_one(parameter)
6     results.append(value)      # hidden shared mutation
```

Prefer functions that receive inputs and return outputs.

```
1 # Better design
2 def worker(parameter):
3     value = run_one(parameter)
4     return parameter, value
```

This makes the data flow explicit and keeps the parallel part easier to test in serial.

A simple scaling model

Parallel speedup is limited by the fraction of the code that can actually run in parallel.

$$S(P) = \frac{T(1)}{T(P)} \approx \frac{1}{(1 - f) + f/P + \text{overhead}(P)}.$$

Here P is the number of workers and f is the parallelizable fraction.

The formula is not a detailed performance model. It is a warning:

- ▶ serial parts limit the maximum speedup,
- ▶ overhead grows when workers communicate or move data,
- ▶ memory bandwidth may saturate before all CPU cores are useful.

Scaling should be measured, not assumed.

Measure scaling, not only runtime

A parallel implementation should be tested with different numbers of workers.

```
1 from time import perf_counter
2 from joblib import Parallel, delayed
3
4 for n_jobs in [1, 2, 4, 8]:
5     t0 = perf_counter()
6
7     results = Parallel(n_jobs=n_jobs)(
8         delayed(run_one)(p) for p in params
9     )
10
11     t1 = perf_counter()
12     print(n_jobs, t1 - t0)
```

Good scaling is not guaranteed. Sometimes two workers help, four workers help less, and eight workers make the program slower.

This is still a useful result: it tells us where the practical limit is.

→ **Example 07 - 05**

Alternative array backends

Some libraries provide NumPy-like array interfaces but use different execution models.

Backend	Typical motivation
CuPy	NumPy-like arrays on NVIDIA GPUs
JAX	compiled array programs, automatic differentiation, accelerators
Dask Array	chunked and possibly distributed array computations
PyTorch	tensor computation, GPUs, automatic differentiation

These tools can be extremely powerful, but switching backend is not just replacing `import numpy as np`. It changes assumptions about execution, memory, data transfer, and supported operations.

CPU or GPU?

A GPU can be very effective when the computation has enough arithmetic work per data transfer and can be expressed in large array operations.

Promising for GPU

- large dense array operations
- large batched linear algebra
- neural-network training
- large stencil-like kernels

Often less promising

- many tiny operations
- frequent CPU-GPU transfers
- complicated Python control flow
- small arrays dominated by overhead

The main question is not whether the GPU is fast. The question is whether enough work can stay on the GPU long enough to compensate for transfer and setup costs.

When not to optimize

Optimization is not always the best use of research time.

It may be better to stop when:

- ▶ the runtime is already small compared with analysis and interpretation time,
- ▶ the optimized code would be much harder to validate,
- ▶ the bottleneck is outside the part of the code we can control,
- ▶ the speedup would not change the scientific workflow,
- ▶ the next improvement would require a large rewrite for a small gain.

A program that runs in ten minutes and is trusted may be more valuable than a program that runs in one minute but is fragile and unreadable.

A final acceleration checklist

Before accepting an accelerated version, ask:

- ▶ Is there a clear baseline?
- ▶ Does the new version reproduce the scientific result?
- ▶ Was the benchmark representative?
- ▶ Was only one major change tested at a time?
- ▶ Is the bottleneck actually reduced?
- ▶ Are memory allocation and data movement understood?
- ▶ Does the speedup survive at the relevant problem size?
- ▶ Can a collaborator still read, run, and modify the code?

Acceleration is successful only if it improves the real scientific workflow.

Lecture summary

The main message of this lecture is a decision process.

baseline → measurement → algorithm → layout → vectorization → memory control
→ JIT → parallel execution → stop

No single technique is the answer to every performance problem.

- ▶ Use algorithmic changes to reduce unnecessary work.
- ▶ Use vectorization when array operations express the computation clearly.
- ▶ Use memory-aware refactoring when temporaries dominate.
- ▶ Use Numba for small numerical kernels and typed loops.
- ▶ Use threads, processes, or joblib only when the workload justifies their overhead.

The best accelerated code is not merely fast. It is measured, validated, and still scientifically understandable.

Section 5

Further reading

Supplementary literature

- ▶ van der Walt, S., Colbert, S.C., Varoquaux, G. The NumPy Array: A Structure for Efficient Numerical Computation. *Computing in Science & Engineering* **13**(2), 22–30 (2011). DOI: <https://doi.org/10.1109/MCSE.2011.37>
- ▶ Lam, S.K., Pitrou, A., Seibert, S. Numba: a LLVM-based Python JIT compiler. In: *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC* (LLVM '15), 1–6 (2015). DOI: <https://doi.org/10.1145/2833157.2833162>
- ▶ Rocklin, M. Dask: Parallel Computation with Blocked algorithms and Task Scheduling. In: *Proceedings of the 14th Python in Science Conference* (SciPy 2015), 126–132 (2015). DOI: <https://doi.org/10.25080/Majora-7b98e3ed-013>
- ▶ Dalcin, L.D., Paz, R.R., Kler, P.A., Cosimo, A. Parallel distributed computing using Python. *Advances in Water Resources* **34**(9), 1124–1139 (2011). DOI: <https://doi.org/10.1016/j.advwatres.2011.04.013>
- ▶ Amdahl, G.M. Validity of the single processor approach to achieving large scale computing capabilities. In: *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference* (AFIPS '67), 483–485 (1967). DOI: <https://doi.org/10.1145/1465482.1465560>